

TRAVAUX PRATIQUES : Installation, Modélisation SQL et Haute Cardinalité sur InfluxDB v3 (Natif VM)

- **Environnement requis** : * Une VM Linux (Debian 13 ou Ubuntu Server) par étudiant/binôme avec accès internet.
 - Droits `sudo` complets sur la VM.
 - Python 3 et `pip` installés sur le système.

Installation Système, Ingestion et Gestion de la Cardinalité

Déploiement Natif et Syntaxe du Line Protocol

Exercice 1 : Installation d'InfluxDB v3 "Bare-Metal"

L'objectif est d'installer proprement le binaire InfluxDB v3 (moteur IOx/Rust) en tant que service système.

1. Ajoutez le dépôt officiel d'InfluxData aux sources APT de votre VM Debian 13.
2. Installez le paquet `influxdb3` (ou téléchargez le binaire officiel compilé pour Rust/Linux x86_64 si vous utilisez la version de la communauté).
3. Configurez le fichier `/etc/influxdb3/influxdb3.conf` pour écouter sur toutes les interfaces réseau (`0.0.0.0`).
4. Créez un service **systemd** (`/etc/systemd/system/influxdb3.service`) si le paquet ne l'a pas fait automatiquement. Activez-le et démarrez-le :

```
sudo systemctl daemon-reload
sudo systemctl enable --now influxdb3
sudo systemctl status influxdb3
```

1. À l'aide de la CLI v3 installée localement, créez une **Database** nommée `usine_analytics`. Générez ensuite un jeton d'authentification (*Token*).

Exercice 2 : Ingestion manuelle et Typage strict

L'ingestion utilise le **Line Protocol**, mais la v3 l'interprète immédiatement sous forme de colonnes de type SQL.

1. Rédigez un fichier texte nommé `data_input.txt` contenant 5 points simulant les capteurs d'une ligne de production. Vous devez inclure des tags (`ligne_id`, `opérateur`) et des fields (`vitesse`, `temperature`, `erreur_critique`).
2. **Contrainte de typage strict** : Forcez le champ `vitesse` en entier (`i`), `erreur_critique` en booléen (`t` ou `f`), et `temperature` en float.
3. Injectez ce fichier en ligne de commande via l'outil `curl` en attaquant directement l'API HTTP locale :

```
curl -XPOST "http://localhost:8086/api/v3/write_lp?db=usine_analytics" \  
-H "Authorization: Token VOTRE_TOKEN" \  
--data-binary @data_input.txt
```

1. *Test de rupture* : Tentez d'injecter immédiatement après un point où `vitesse` est envoyé sans le suffixe `i` (Float). Analysez le code d'erreur HTTP renvoyé par le moteur en Rust face au conflit de type.

Exercice 3 : Premier requêtage en SQL ANSI via la CLI

1. Lancez l'outil interactif de requêtage d'InfluxDB v3 en mode SQL depuis votre terminal :

```
influx3 sql --database usine_analytics
```

1. Rédigez une requête SQL classique pour lister toutes les lignes distinctes de votre table.
2. Appliquez un filtre `WHERE time >= NOW() - INTERVAL '15 MINUTES'`.

Après-midi : Comportement face à la Haute Cardinalité

Exercice 4 : Injection de données massives et éphémères

Nous allons stresser le stockage colonnaire de la v3 en générant une quantité infinie de tags uniques (haute cardinalité).

1. Installez la bibliothèque cliente Python 3 native pour la v3 au niveau du système :

```
pip install influxdb3-python --break-system-packages
```

1. Rédigez un script Python `stress_test.py`. Ce script doit injecter 5 000 points par seconde dans la table `flux_production`.
2. Chaque point doit impérativement inclure un tag nommé `piece_serial_number` contenant un **UUID unique (chaîne aléatoire) généré à chaque itération**.
3. Lancez le script en tâche de fond (`python3 stress_test.py &`).

Exercice 5 : Analyse de l'impact OS et des fichiers Parquet

Pendant que le script s'exécute de manière intensive sur la VM :

1. Surveillez en temps réel la mémoire RAM du processus InfluxDB v3 via la commande `htop` ou `top`. Constatez la stabilité de la RAM malgré les millions de tags uniques créés.
2. Allez inspecter le répertoire de stockage physique des données (généralement situé dans `/var/lib/influxdb3/`).
3. Identifiez la création des fichiers journaux temporaires (**WAL**), puis observez l'apparition des fichiers de stockage finaux au format standard du Big Data : **Apache Parquet** (`.parquet`).
4. Rédigez un paragraphe expliquant pourquoi la v3 tolère cette haute cardinalité sans s'effondrer par rapport aux index inversés des anciennes générations.

Requêtage Analytique, Downsampling et Restitution

Maîtrise des fonctions temporelles SQL et Data Skipping

Exercice 6 : Requêtes analytiques avancées

1. Ouvrez à nouveau la CLI SQL d'InfluxDB v3.
2. Rédigez une requête SQL utilisant la fonction native de la v3 `TIME_BUCKET(INTERVAL '5 MINUTES', time)` pour calculer la moyenne de température de la ligne de production par tranches de 5 minutes.
3. Rédigez une requête SQL avancée calculant l'écart-type de la vitesse des pièces, regroupé par `opérateur` et par heure (`GROUP BY opérateur, TIME_BUCKET(...)`).
4. Effectuez une requête visant à chercher un `piece_serial_number` spécifique (UUID) au milieu de la masse de données. Notez le temps d'exécution.

Exercice 7 : Comprendre le Data Skipping

1. À l'aide de la clause SQL `EXPLAIN` placée juste devant votre requête précédente, analysez le plan d'exécution généré par le moteur.
2. Identifiez comment le moteur de requêtage (*Apache Arrow DataFusion*) utilise les métadonnées légères des fichiers Parquet (Min/Max temporels et valeurs de colonnes) pour éliminer les fichiers inutiles avant la lecture sur disque (*Pruning / Data Skipping*).

Cycle de vie et Industrialisation

Exercice 8 : Pipeline de Downsampling en SQL Pur via Cron

Dans InfluxDB v3, les tâches planifiées de sous-échantillonnage s'exécutent via des requêtes SQL standard pilotées par le système.

1. Créez une seconde Database nommée `usine_historique_5ans`.
2. Écrivez un script Python ou Bash `downsample.py` contenant une requête SQL de transfert de données :

```
INSERT INTO usine_historique_5ans.table_horaire
SELECT TIME_BUCKET(INTERVAL '1 HOUR', time) AS time, AVG(temperature),
MAX(vitesse)
FROM usine_analytics.flux_production
WHERE time >= NOW() - INTERVAL '1 DAY'
GROUP BY 1;
```

1. Planifiez l'exécution de ce script de manière récurrente dans le gestionnaire de tâches natif de la VM : le **Crontab** de Linux (**crontab -e**).
2. Validez le bon transfert en inspectant la base historique.

Exercice 9 : Visualisation Grafana en natif

1. Installez le serveur Grafana directement sur la VM via le gestionnaire de paquets APT (**sudo apt install grafana**). Démarrez le service systemd associé.
2. Connectez-vous à Grafana (Port **3000**).
3. Ajoutez InfluxDB v3 comme source de données en utilisant le connecteur officiel **InfluxDB SQL (Apache Arrow / Flight SQL)**.
4. Créez un Dashboard de supervision industrielle affichant :
 - Le débit de production en temps réel (données chaudes de la table brute).
 - L'historique des tendances sur 24h issu de votre table de downsampling (base historique).